

Improve and Estimate Real Estate Returns

Susmita Karyakarte
New York University
New York, USA
sk8663@nyu.edu

Abstract

The Improve and Estimate Real Estate Returns System is for anyone who seeks to invest in properties to maximize their return on their investment, which is achieved via building a predictive model for real estate price prediction and generating counterfactual explanations to increase the return on the price. The reason why this application is interesting it offers a simple dashboard where a user can enter a certain set of feature values such as the neighborhood and borough of the property, and some basic information about the building or property of the user which involves the total number of residential units or square feet area of the property and it provides the user with interactive visualizations showing an explanation of the features affecting the current or future price of the property. Currently the system is able to predict real estate prices with an accuracy of 73%.

1 Introduction

There is no denying that New York is one of the most expensive places to own or rent a place. NYC Real Estate has always been a popular topic of discussion, with prices rising high at a rate of a minimum of 10 % every year. The goal of this project involves coming up with a one place for all system for anyone who seeks to invest in properties and maximize their return on this investment, through building a predictive model for real estate price prediction, explaining the model and generating counterfactual explanations to increase the return on the price. In order to build the system I will be using the NYC Real Estate market data given its popularity and hence the easy availability of the datasets. Major real estate analytics firms such as Zillow, Stratodem, Real Capital Analytics and many more are working on enhancing their price prediction systems which already provide pretty good estimates of the prices of properties in accordance to the current market rates. With widely available data sources in reference to property sales and transactions data firms are now presented with the problem of having too much data and not knowing how to utilize it to its full extent and the various applications that can be built on top of it that would in turn help the general buying and selling public.[1] One such application that can be built on top of the existing data is a system that not only explains the reasons for a certain price prediction but also helps to analyze what can be done to achieve a certain price point for a property in order to maximize profit on the investment which is what makes

developing this application interesting. A web app that provides a predictive model, it's explanation, counterfactuals and a visual analytic component all in one place for any user. Any developer who wants to build on top of the system can easily make use of the app source code which is simplistic and ready to get started with, and add more data for building the model or make use of LIME or SHAP for various tasks and display the results on the frontend of the app.

2 Related Work

The Rolling Sales Dataset for NYC provided by NYC Open Data is a very popular dataset for exploring data analysis techniques and getting practice on building predictive models. There are numerous notebooks available on Kaggle and GitHub which have preprocessed the data and performed EDA with the help of numerous visualizations which also definitely helped me when I was working on my predictive model. Specifically one such repository which is available on [GitHub](#) [4] which does not build on top of the NYC Open Data but rather data presented by a Real Estate Investment Trust Firm (REIT) motivated me to go forward with this project. The data presented by the firm along with the property attributes consisted of demographic attributes about the buyer of the properties as well and the predictive model built on top of this model had an accuracy of more than 95 %. Now everyone doesn't always have access to data which has been collected by anyone professionally and being interested in the market I wanted to make use of the easily available resources to me and make something useful which can then be later improved upon.

While there have been efforts to understand and interpret complex machine learning models by local approximation of the models or by ranking of features in the model by libraries such as LIME and SHAP, DiCE presented a new approach of interrogating a ML model to find required changes that would flip the model's decision. It provides this information by showing the feature-perturbed versions of the same input who would have got a different outcome. Most explanation techniques face an inherent tradeoff between fidelity and interpretability: a high-fidelity explanation for an ML model tends to be complex and hard to interpret, while an interpretable explanation is often inconsistent with the ML model it was meant to explain. Counterfactual explanations offer a promising alternative. CF explanations from DiCE are also useful to the decision-maker who can use them to evaluate the trustworthiness of a particular prediction from the ML

model. Similarly, CF explanations over multiple inputs can be useful for decision-evaluators to evaluate criteria such as fairness, and by model developers to debug their models and prevent errors on new data. [6]

3 Approach/Methods

3.1 Data Acquisition

For building the predictive model I used the data that has been provided by the NYC Department of Finance titled [Rolling Sales \(Annualized Sales\) Data](#) [8]. The NYC Department of Finance provides yearly data of all sales of properties in all the five boroughs. These properties involve all classes of building categories ranging from one family houses to fireproof warehouses, co-op investments and more. Naturally given different type of buildings these properties have different tax classes as well. These properties are divided into four different tax classes as mentioned below

- **Class 1:** Includes most residential property of up to three units (such as one-, two-, and three-family homes and small stores or offices with one or two attached apartments)
- **Class 2:** Includes all other property that is primarily residential, such as cooperatives and condominiums.
- **Class 3:** Includes property with equipment owned by a gas, telephone or electric company.
- **Class 4:** Includes all other properties not included above such as offices, factories, warehouses, garage buildings, etc.

Some more information on the data is provided [here](#) [9]. Apart from the location information the data consists of information about the total units of the property (residential or commercial) that exist at the location and the total area of the property. For building the predictive model I acquired the data for the years 2016-2021.

3.2 Preprocessing

I firstly went through each of the 30 data files and explored the data. Then quickly realized that while the order of the information contained in each column remained the same, the column names differed a little in the files year by year. I made all the column names consistent within the data as I started with the preprocessing. It also involved data types conversions for different columns and dropping of information such as the ADDRESS, ZIPCODE, BLOCK, LOT and EASEMENT features. Since these features were categorical I rather decided to use the BOROUGH and NEIGHBORHOOD features to represent these data points and reduce the number of categories.

Eventually the features I decided to go forward with in order to build the model involved BOROUGH, NEIGHBORHOOD, TAX CLASS, BUILDING CLASS, RESIDENTIAL UNITS, COMMERCIAL UNITS, TOTAL UNITS, LAND SQUARE

FEET, GROSS SQUARE FEET, YEAR BUILT and SALE PRICE of the properties. This is the same information the system will ask the user to enter in order to return the price. For simplicity the model currently assumes that TOTAL UNITS is the addition of the number of RESIDENTIAL UNITS and COMMERCIAL UNITS.

After I merged all the data, as the next step, I checked for NaN and null values, especially on the 'SALE PRICE' column. I used the `dropna()` and `isnull()` functions for this purpose. Furthermore, when I checked the unique values for each column, it consisted of '-' values for most of the rows that indicated missing data. There was a lot of missing data and also invalid data which I took care of during data preparation.

A major step in preprocessing involved creating new features out of the categorical features and the YEAR BUILT feature. I calculated the age of each property out of the year in which the property was built and saved it as the PROPERTY AGE feature for the model. In order to make efficient use of the categorical features I decided to go forward with the target encoding strategy and target encoded the BOROUGH, NEIGHBORHOOD, TAX CLASS and BUILDING CLASS features out of the SALE PRICE of the property.

I did so by grouping the properties by the corresponding feature class and calculating the standard deviation of the sale price for that class within the data and then replacing the feature values with the calculated standard deviation of the sale prices. This preprocessing approach showed a performance improvement in the accuracy of the model by at least 7%. After all the preprocessing I noticed there are no properties having the TAX CLASS as 3 within the samples that are further used for building the model hence the model currently can only predict prices for the rest three classes of the properties.

3.3 The Predictive Model

For modelling the sale price of the properties I considered an assortment of models to observe the performance of every model and go with the best technique. I started out with building a simple linear regression model which provided a very low performance of about 33% on just the training data hence I decided not to go forward with the regression technique. Next I tried building a random forest model which gave me a performance of 93 % on the training data and 73% on the test data. Hence as I expected an ensemble method proved to be useful for the current use case of data.

In an effort to improve performance I thought of trying a boosted ensemble method to build the model and went ahead with building a LightGBM model assuming that it would handle categorical features and outliers better than random forest however it returned an even lower score of 82 % on the

training data and a score of 67 % on the test data. Therefore as random forest proved to be the best among these, I chose to move forward with the technique for building the system and also to generate explanations for.

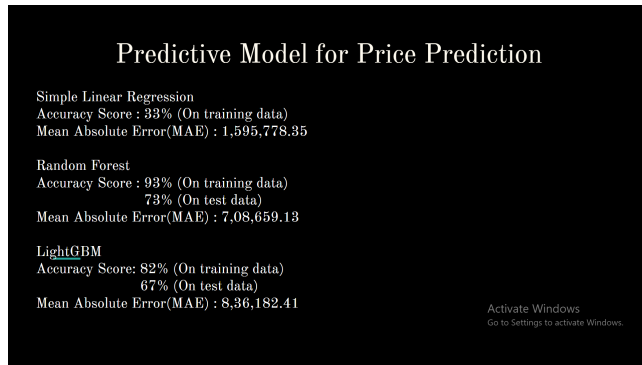


Figure 1. Predictive Model Techniques

3.4 The Dashboard

The basic idea of the system was an application that provides the user with a predictive model that user can use to predict current and future price of a property and along with the price values get an explanation of which features were affecting the price the most. Then the second use of the system would be to generate counterfactuals that would tell the user the values of the features that would be required to reach a certain price point. And lastly provide the user with an interactive map that would show similar properties which were similar in terms of their shap values.

In order to build such a dashboard I started out with using the React and Flask technologies for creating the app. However I found myself spending a lot of time on creating the frontend which involved writing CSS for every individual element of the page consisting of different forms, visualizations and maps, even linking all of my backend elements to the frontend via API endpoints. I realized this was straying me away from working on the main functionalities of the app.

Eventually after even trying out a few other technologies like Dash, Plotly and JavaScript frameworks, I started learning the Streamlit framework which enabled me to focus less on my frontend elements and connecting them to a server and allowed me to continue my work in Python and simply display all of it in the form of a web app which can also be readily deployed to the cloud. All it involved was include my Python code in a GitHub repository as usual and after installing all the dependencies Streamlit enables us to share our app by generating a public URL for our app [13]. I made use of the multi-page app framework provided here [2] as

currently Streamlit mostly supports single-page web applications and it requires certain hacks in order to implement multi-page web applications. The only limitation of Streamlit is that it takes some time to load all the application data the first time that it runs although for further application runs the data gets cached which improves the performance and running time of the application.

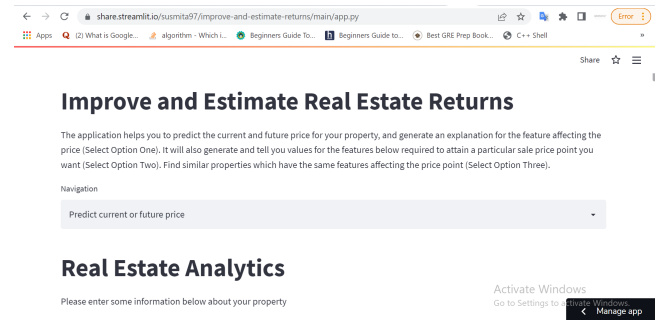


Figure 2. The Dashboard

To summarize the application can be thought of as three parts.

- **Part One** predicts the current and future price for your property, and generate an explanation for the feature affecting the price. If the user wants a future price point the user can add the number of years from now until the estimated year in the Property Age value. More about this discussed below.

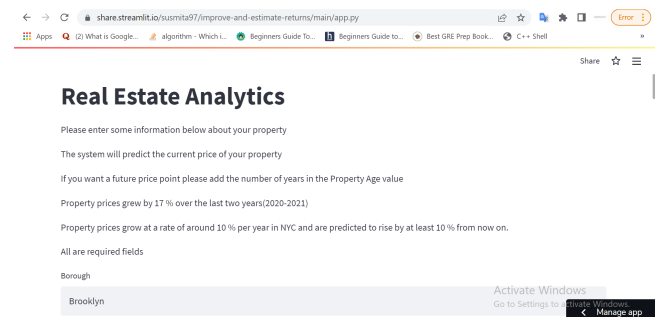


Figure 3. Part One of the Application that predicts the Price and generates Explanations

- **Part Two** generates and tells you values for the features required to attain a particular sale price point you want(Counterfactuals)
- **Part Three** finds similar properties which have the same features affecting the price point(have similar shap values)

Figure 4. The Form for the User

Figure 5. Controls for generating LIME Explanation

4 Results

4.1 Computing LIME Explanations

For an instance of the features given below:

BOROUGH : Manhattan

NEIGHBORHOOD : Alphabet City

RESIDENTIAL UNITS : 3

COMMERCIAL UNITS : 0

LAND SQUARE FEET AREA : 3000

GROSS SQUARE FEET AREA : 4000

PROPERTY AGE : 120

TAX CLASS OF THE PROPERTY : 1

BUILDING CLASS CATEGORY : Three Family Dwellings

The model predicted a value of \$ 7,146,280.65 as the sale price. Before looking at the local explanation generated by LIME I produced a plot that showed the global contribution of each of the features also shown below.

The plot shows GROSS SQUARE FEET area feature being the most important out of all the features in determining the sale price. The next most important feature is the NEIGHBORHOOD of the property. These results fall in line with the general notion of the factors that affect the price of any real estate entity are usually the location and area of the property. Our model seems to be giving more importance to area of the property than the location which also makes sense as when observed the median prices of properties over all five

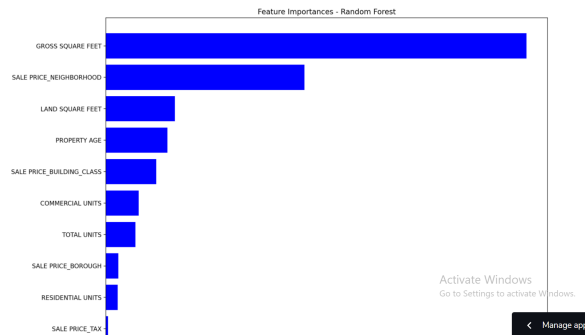


Figure 6. Global Feature Importances

boroughs don't have a huge difference, in comparison to say the prices of properties within NYC or upstate New York or any suburb within the US.

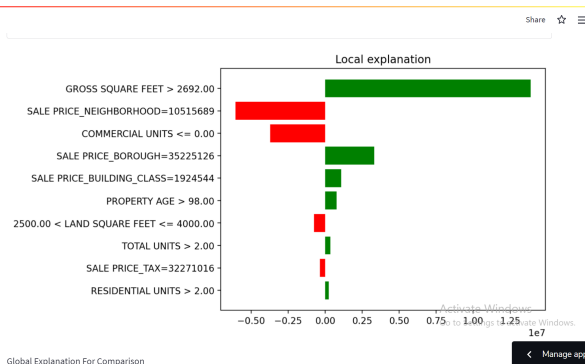


Figure 7. Local Feature Explanation by LIME

Shown above is the explanation of the instance generated by LIME which shows GROSS SQUARE FEET area feature affecting the price of the property the most.[10] In the explanation generated above NEIGHBORHOOD of the property seems to be affecting the price of the property negatively. The properties within the NEIGHBORHOOD of Alphabet City have sale prices lesser than neighborhoods like say the Upper West or Tribeca within Manhattan therefore within a neighborhood of 1500 samples the explanation that neighborhood reduces the property value isn't far away than the factual explanation. When I changed the neighborhood value for the same above instance to UPPER WEST SIDE which is one of the most expensive neighborhoods within NYC it generated the following explanation.

This explanation was generated for the same number of features and number of samples as above. We can see above that now the location of the property is having a positive effect on the price of the property. The price predicted for the property for this instance is \$ 15,778,474.22 which falls in line with the current prices for the properties in this area. Therefore this explanation helps to identify that Alphabet

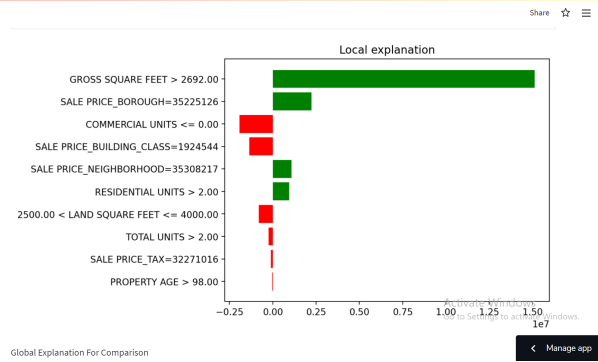


Figure 8. Local Feature Explanation by LIME for a different neighborhood for the same instance

City is one of the least expensive neighborhoods within Manhattan. If any explanation shows a negative value for either the BOROUGH or NEIGHBORHOOD features then we have identified a cheaper location for our property hunt.

Usually presence of commercial units within the property helps to affect the price of the property positively hence here we can see it does not help the property's price. We can also see that if the LAND SQUARE FEET area of the property is less than 4000 then it affects the price negatively and older the property although not much but the age of the property has a positive effect on the property. If the total number of units are more it usually shows a positive effect on the price even within a neighborhood of samples. An interactive component that allows the user to change the number of features and number of samples used to generate the explanation has been added to the application to make the system more interactive as can be seen in Figure 5.

4.2 Computing SHAP values

After preprocessing all of the data my model was trained on about 119,800 samples of property transactions. I split the data in a 80:20 split with 80 % of the data being my model's training data and using the rest as test data. It was not possible to generate SHAP values for all of the test data that consisted of about 24,000 transactions due to time constraints. Hence, I precomputed a subset of 10,000 SHAP values from my test data and saved it on my computer as computing all of the values while running the application was also not feasible. Computing these SHAP values was necessary as it was required by the third part of the application which consisted of finding similar properties based on their SHAP explanations. I trained a TreeExplainer on the random forest model in order to compute the SHAP values. [5]

Before providing the SHAP explanation for the individual instance I produced a SHAP summary bar plot of all the 10,000 precomputed SHAP values that gives us an idea of the

features that SHAP currently considers the most important in arriving at the predicted price.

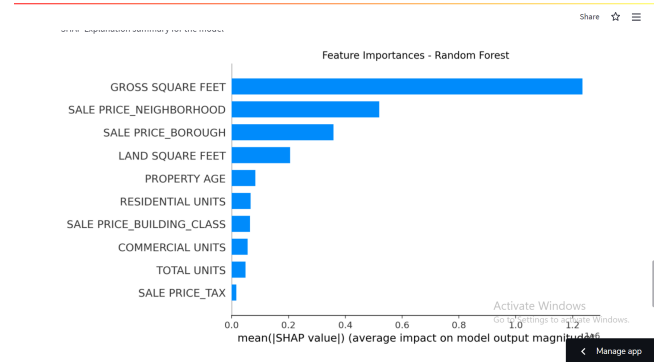


Figure 9. SHAP Summary Bar Plot

SHAP seems to be producing the same results as LIME and considers GROSS SQUARE FEET and NEIGHBORHOOD as the most important features in arriving at the current sale price.

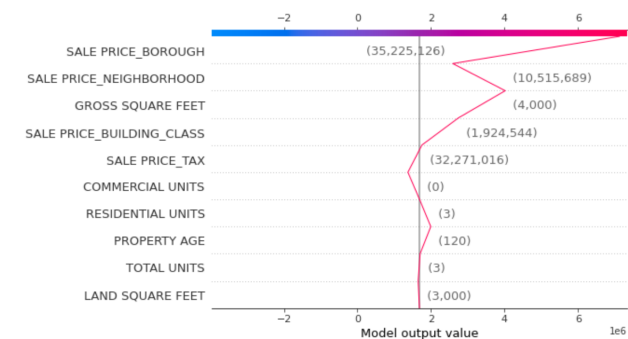


Figure 10. SHAP Decision Plot for individual instance explanation

After the summary bar plot, I decided to display a decision plot for the local explanation of the output. A decision plot adds the feature values and shows how complex models arrive to a decision for producing the predicted value. The x-axis represents the model's output. All SHAP values represented are relative to the model's expected value. Here the expected value is the mean of the raw model predictions for the training data. To obtain raw prediction values, the expected value is added to the sum of SHAP values for each observation. The y-axis lists the model's features. By default, the features are ordered by descending importance. The importance is calculated over the observations plotted. This is usually different than the importance ordering for the entire dataset. [12] Hence I decided to plot a decision plot for the explanation. Decision plots have a lot of uses which

involve detecting outliers, comparing predictions from different models and many more which may prove to be useful in future development of the app.

In the plot produced we can see for this particular instance SHAP shows that the borough, neighborhood and gross area of the property are the most important features in arriving at the predicted price. The ordering of the features in their importance for the local explanation here is a little different than the global explanation however it still reasonable and understandable.

When I looked at the data that was used a subset for generating the precomputed SHAP values most of the data points turned out to be properties located within Manhattan and the rest were located within Bronx. Hence the third part of the application can currently only find similar properties which will be present within these two boroughs.

4.3 Computing Counterfactuals and Working with DICE

The second part of the application is responsible for counterfactual generation. As can be seen in the figure below the second part of the app provides the user with the same form again with the change of the desired price range. In the input boxes the user can provide the current price of the property and the user's desired property price and on submission the app generates a maximum of 20 counterfactuals which help in telling the user which values can be changed in order to reach the desired range of property price.

Figure 11. SHAP Decision Plot for individual instance explanation

The DICE library provides three different methods which can be used for generation of counterfactuals and they involve "random", "genetic" and "kdtree". Since all the methods are model agnostic I experimented with all three methods however the method which returned a result that seemed to be making sense was kdtree. Within the generate_counterfactuals method provided by the library we can specify the number of counterfactuals we want, the desired range of our property price and also the permitted ranges of values for our features.[7] The permitted_ranges parameter within the

method proved to be the most useful in generating counterfactuals as it keeps the returning values within a range that is feasible and can be changed in the real world.

For example if the permitted range is not set for the PROPERTY AGE feature then the counterfactuals returned for the current instance of data actually involved all rows consisting of values less than 120 years of property age which was not possible in reality. Hence by playing around with these values we can actually reach a conclusion which is of help to us. Counterfactuals are not generated here for the categorical features as we cannot really change the location of the property, also changing the building class category or tax class would not be very feasible in the real world.

Your property attributes currently are						
	RESIDENTIAL UNITS	COMMERCIAL UNITS	TOTAL UNITS	LAND SQUARE FEET	GROSS SQUARE FEET	PROPERTY AGE
0	3.0000	0.0000	3.0000	3,000.0000	4,000.0000	120.0000

The generated counterfactuals						
	RESIDENTIAL UNITS	COMMERCIAL UNITS	TOTAL UNITS	LAND SQUARE FEET	GROSS SQUARE FEET	PROPERTY AGE
0	0.0000	1.0000	1.0000	4,437.0000	6,000.0000	120.0000
1	0.0000	1.0000	1.0000	2,011.0000	8,283.0000	120.0000
2	8.0000	5.0000	13.0000	7,510.0000	23,324.0000	129.0000
3	0.0000	1.0000	1.0000	1,787.0000	3,360.0000	194.0000
4	4.0000	1.0000	5.0000	2,317.0000	10,425.0000	120.0000
5	22.0000	2.0000	24.0000	3,120.0000	11,400.0000	120.0000

Figure 12. Counterfactuals for the instance

The results that are returned consist of a table showing the generated counterfactual values and the table is followed by graphs that visually display a change in the current value and the generated counterfactuals for all the features. Usually out of the generated counterfactuals there are only one or two CFs which are of use to the user.

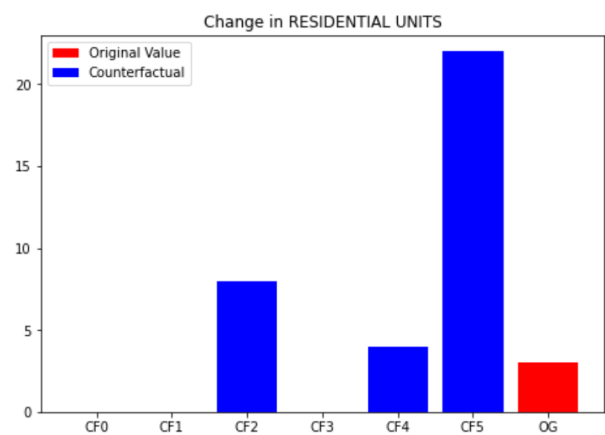


Figure 13. Change in RESIDENTIAL UNITS

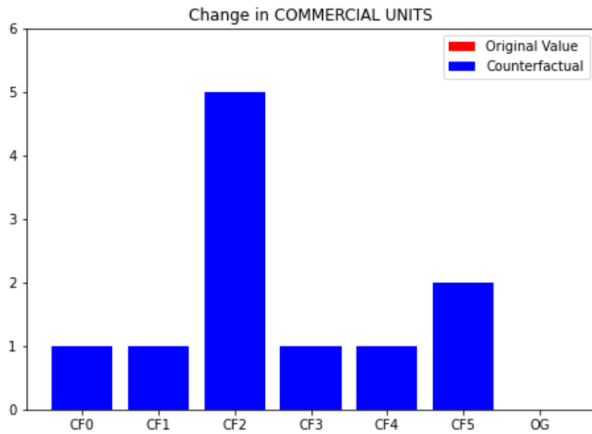


Figure 14. Change in COMMERCIAL UNITS

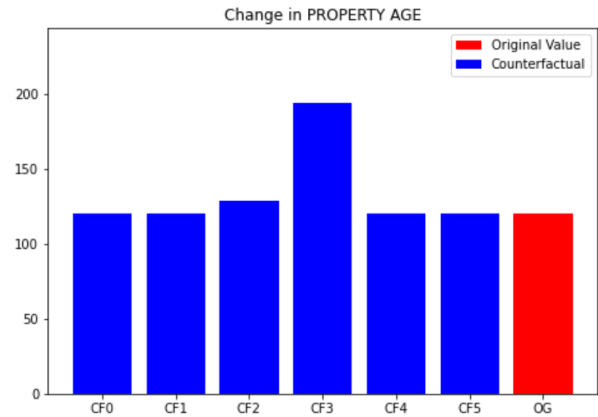


Figure 17. Change in PROPERTY AGE

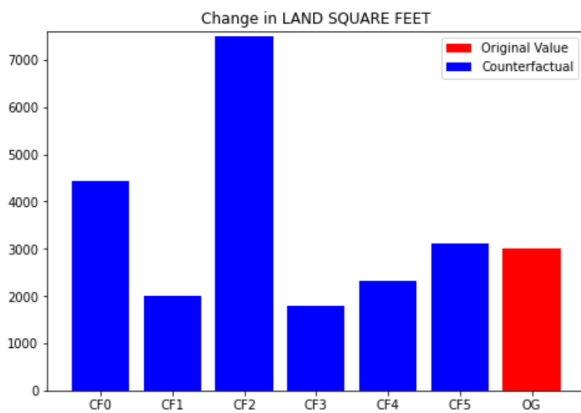


Figure 15. Change in LAND SQUARE FEET

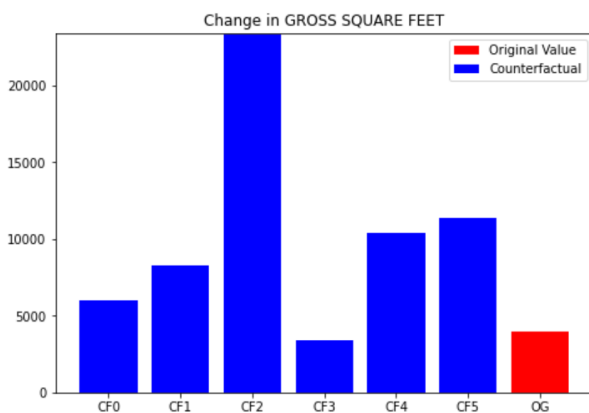


Figure 16. Change in GROSS SQUARE FEET

As shown in Figure 11 we try to figure out what can be done to double the current price of the property and generate

CFs in order to learn about the same. We can see from the results that one particular row has values for all the features where TOTAL UNITS are 13 with a substantial increase in the area of the property and it says that after 9 years a price within the desired range can be achieved. The prices in NYC have over the years shown a steady increase with about 10 % increase in the prices every year and they are projected to rise by 17 % over the year. The model does not take into account the SALE DATE variable within the data for predicting the price. Once this is taken into account the model and even the generated CFs explanations will be more helpful than they currently are. If a blue or red bar within the visualizations shown doesn't exist it indicates a value of 0 for the given feature.

4.4 Interactive Visualization with Leafmap

The third part of the application involves finding similar properties which have the same explanation of features contributing to the property price produced by SHAP. In order to group these properties together and form clusters of these data points I decided to run a clustering algorithm like K-Means on the precomputed SHAP values and find the optimal number of clusters. The advantage of using shap values for clustering is that shap values for all features are on the same scale.[11] In order to find the optimal number of clusters I used the elbow method and plotted a curve as shown in Figure 18. The curve starts to run in a straight line and form the elbow shape around the value of 4 so I decided to use 3 as the optimal number of clusters.

After running the K-Means algorithm on the precomputed SHAP values, I divided the 10,000 SHAP values along with their corresponding data points from the test data into 3 clusters. I observed that the algorithm was grouping most observations, at least 8000 of them, in a single cluster which made sense as the values consisted of transactions from only

Interactive Map Visualization

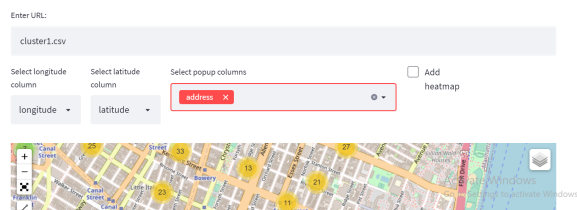


Figure 18. Interactive Visualization

two boroughs hence the distance between the observations in Manhattan was definitely smaller than those in Bronx and the algorithm was grouping them together. Also the data consists more of transactions in Manhattan than in Bronx and therefore one of the rest two clusters involves transactions mostly from Bronx. I ran the algorithm two times again on the first cluster which returned good results on the second run however decided not to go forward with the results due to time constraints. I could have also grouped the transactions within Manhattan and Bronx together before itself but it defeats the purpose of showing similar properties all over NYC. The remaining SHAP values can still be computed later on and simply fitted into the existing code for better results. I built the component on top of a framework provided [here](#) [3] for use of leafmaps for the system.

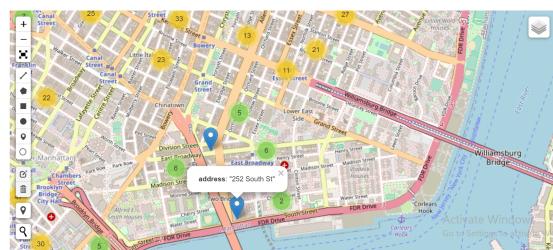


Figure 19. Properties with similar SHAP values with Popup

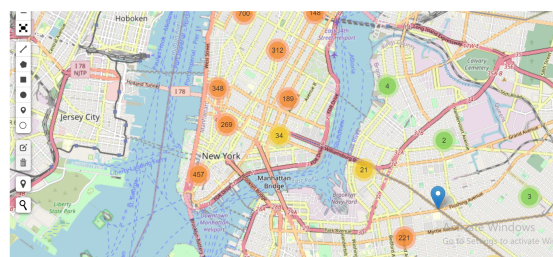


Figure 20. Properties with similar SHAP values

One of the problems I observed with rendering of the Leafmap Visualization on the Streamlit platform was that it

cannot handle a whole lot of to be plotted data points alongside the associated huge data for the popup of the markers. I wished to display all the corresponding property feature values for every property on the map but that would either make the application crash or for the map to simply not display or work on the page. Hence I decided to add an element on the page which lets the user to decide the information that interests them and then display only those features on the map. Another interesting feature of the provided visualization here would be that it is a free map exploration tool, any user can upload their data onto the page they want and as long as the file has a list of location coordinates the user can explore any data points they want. Leafmap maps itself provide users a lot of functionalities that any user can explore and make use of through the system.

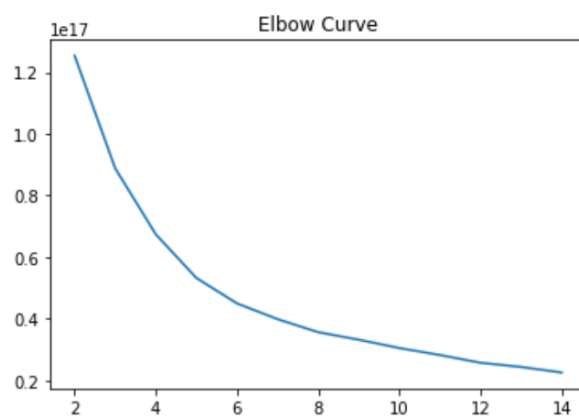


Figure 21. Elbow Curve

5 Discussion

5.1 Implications of the Results/Analysis

Overall the performance of the application heavily depends on the Streamlit platform and how the platform deals with loading of the data. The application uses a lot of time to load data and perform computations the first you start the application despite the fact that the data being passed to the application and rendered on the frontend is already pre-processed and even most values are already precomputed. It takes about at least 40 seconds to load the neighborhood values to be loaded in the form after the user selects a borough value from the drop down in the form and takes about 20 to 30 seconds to return results for the form for the first two parts of the application. Although Streamlit caches the data for further runs of the application and that does help a smoother use of the app a user is still estimated to spend about 7-9 minutes on the app if they want to make full use of all the functionalities every time they try to access it. This

will be a problem if the application were to be scaled in the future. Despite this the application still provides all its functionalities very well from managing invalid inputs to the form or even producing outputs that will always be reasonable, intuitive and understandable to any user of the system. The visual elements of the system work perfectly in explaining the results being returned by the system providing an interactive element for the user.

5.2 Limitations and Future Work

Usually any real estate predictive models rely heavily on information about the properties and their location and the prices are also at the same time affected by a great extent by the surroundings of the properties. Possible future work for the project can definitely involve adding more data about the surroundings of the properties which is easily possible by calculating the distance to nearby parks, businesses, grocery stores, cafes and subway stations. The list of all of these places is available on the NYC Open Data page which also includes the coordinates to all these places over all the five boroughs. Just how the model showed a performance improvement of 7 % just by target encoding the variables, adding more information about these places would definitely improve the accuracy of the model. Currently the only measure used for evaluation of the model is the accuracy score and mean absolute error (MAE). More work can be done on choosing evaluation metrics for the model.

Currently the model does not take into account how the prices have changed over the years. There has been a steady increase in the median prices of properties in NYC since 2016 with an increase of 6.75 % from 2016 to 2017 all the way up to a 17% increase from 2020 to 2021. Since the SALE DATES of the transactions were not taken into account the model might not be very accurate in predicting future prices which can be improved upon. More SHAP values can be calculated for all of the data for finding similar properties and better clustering algorithm or more iterations of the K-Means algorithm will help to find properties in a much better way.

6 Conclusion

The web app successfully provides its users with a system that anyone can use to get more information on how real estate prediction systems work and the features that drive the property prices and their contribution. The data used in making the project was simple open source data that is available to anyone who possibly wants to explore the field more or even build commercial products for the real estate agents. The web app's source code and functionality being open source can be accessed by anyone and can act as a starting point for building more developed products and even better predictive models by adding more variables

like SALE DATE or even more categorical variables like the ADDRESS and ZIP CODE to make the model predictions more accurate. The web app currently serves the purpose of providing it's users more insight into the real estate market but given the scope of improvement it is capable of doing much more.

References

- [1] Doshi S. Means J. Sanghvi A. Asaftei, G. M. 2018. *The potential in real estate analytics*. McKinsey. Retrieved 12 May 2022 from <https://www.mckinsey.com/industries/real-estate/our-insights/getting-ahead-of-the-market-how-big-data-is-transforming-real-estate>
- [2] dataprofessor. 2021. *multi-page-app*. Retrieved 12 May 2022 from <https://github.com/dataprofessor/multi-page-app>
- [3] giswqs. 2021. *streamlit-geospatial*. Retrieved 12 May 2022 from <https://github.com/giswqs/streamlit-geospatial/blob/master/apps/xy.py>
- [4] jsrpy. 2018. *NYC-Property-Regression*. Retrieved 12 May 2022 from <https://github.com/jsrpy/NYC-Property-Regression>
- [5] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. 2020. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* 2, 1 (2020), 2522–5839.
- [6] Microsoft. 2019. *DiCE: Diverse Counterfactual Explanations for Machine Learning Classifiers*. Retrieved 12 May 2022 from [MachineLearningClassifiers](https://www.microsoft.com/en-us/research/project/dice/).Microsoft.<https://www.microsoft.com/en-us/research/project/dice/>
- [7] Ramaravind K Mothilal, Amit Sharma, and Chenhao Tan. 2020. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*. 607–617.
- [8] NYCGov. 2022. *Annualized Sales Update*. Retrieved 12 May 2022 from <https://www1.nyc.gov/site/finance/taxes/property-annualized-sales-update.page>
- [9] NYCGov. 2022. *Glossary of Terms*. Retrieved 12 May 2022 from https://www1.nyc.gov/assets/finance/downloads/pdf/07pdf/glossary_rsf071607.pdf
- [10] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*. 1135–1144.
- [11] SaadAhmed. 2020. *Identifying High-Risk Groups Using SHAP Values on Healthcare Data*. Retrieved 12 May 2022 from <https://towardsdatascience.com/identifying-high-risk-groups-using-shap-values-on-healthcare-data-e3e7198f30f6>
- [12] SHAP. 2018. *Basic decision plot features*. https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots/decision_plot.html#Basic-decision-plot-features
- [13] Susmita. 2022. *improve-and-estimate-returns*. Retrieved 12 May 2022 from <https://share.streamlit.io/susmita97/improve-and-estimate-returns/main/app.py>